2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER) | 979-8-3503-3066-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/SANER60148.2024.00057

Comparative Study of Reinforcement Learning in GitHub Pull Request Outcome Predictions

Rinkesh Joshi
Carleton University
Canada
rinkeshjoshi@cmail.carleton.ca

Nafiseh Kahani
Carleton University
Canada
kahani@sce.carleton.ca

***Abstract*—In the rapidly evolving field of software development, pull-based development models, facilitated by tools such as GitHub, are essential for collaboration. This study explores factors that influence pull request (PR) outcomes and employs two Reinforcement Learning (RL) formalizations, modeled as Markov Decision Processes, for PR outcome prediction. The first model leverages 72 PR features and achieves a G-mean score of 0.82664, while the second focuses solely on PR discussions, resulting in a G-mean of 0.88372. Using a specially designed reward function, these RL formalizations strategically address data imbalance and excel in mimicking both single-stage and multi-stage PR review processes. They outperform baseline models (Random Forest, XGBoost, and a Naive Bayes baseline) across various data splits—namely 80/20, 50/50, and 20/80—and are particularly effective at predicting PR rejections. The study also makes its datasets publicly available for future research.**

***Index Terms*—Software Engineering, Pull-based Software Development, Distributed Software Development, Pull Requests, Code Review, GitHub, Pull Request Outcome Prediction, Predictive Models, Reinforcement Learning, Sentiment Analysis**

I. Introduction

The emergence of Distributed Version Control Systems (DVCS) has led to the increase in adoption of the pull-based software development model, especially in Open Source Software (OSS) development [1]. GitHub, is the primary enabling platform for pull-based development model, where developers (contributors) modify an isolated copy of the project's repository, and then submit a Pull Request (PR) to the repository owners. The owners (integrators) then evaluate the suggested changes in PR through an interactive review process, and they either *approve* or *reject* the proposed merge into the main codebase of the project [2].

During the review process, other individuals from the development community can also actively engage in discussions, offering insights into the proposed changes. Also, Development Operations (DevOps) tools enhance this process by automatically assessing code compatibility and adherence to standards [3]. The collaborative nature of this process has transformed GitHub into an active ecosystem, with over 100 million developers, 420 million total projects, and over 4.5 billion contributions in 2023 alone [4].

The efficiency of this review process, often measured by code velocity, is crucial for both the developers and open source projects [5], [6]. Delays in PR reviews can lead to a variety of issues, such as communication breakdowns and complicated merge conflicts [7]. Several studies have investigated factors influencing PR outcomes and merge times [1], [2], [3], [5], [8], [9], [10], [11], [12], [13], [14]. These studies have explored contributors' and integrators' perspectives, presented PR prioritization tools, and proposed effort estimation models [7], [15], [16], [17], [18], [19], [20], [21]. Additionally, they have delved into the code review process, participation factors, and prediction models for PR review outcomes [6], [22], [23], [24], [25], [26], [27], [28], [29].

To complement the above studies, and inspired by the successes of Reinforcement Learning (RL) in various domains such as computer gaming, recommendation systems, prioritization systems, real-time ad bidding [30], as well as in source code summarization [31], we have adopted RL for predicting PR outcomes in this study. We have formalized the task of PR review outcome prediction as a sequential decision-making problem, which allows us to capture the dynamic interactions between the PR environment and a decision-making agent using RL. This interactive approach holds promise for developing predictive models capable of handling large datasets and extensive feature sets, enabling us to gain a more comprehensive understanding of the nuances of the PR review process. Our study introduces two distinct RL formalizations, each built on large-scale datasets. One formalization considers a wide range of PR-related features, while the other focuses exclusively on PR discussions. More specifically, our study answers the following research questions:

RQ1: *How feasible and effective are RL approaches in predicting outcomes of PR reviews?*

This foundational question guides our investigation into assessing RL formalizations in the context of PR review decision-making. It delves into the specifics of formalizing the PR environment and designing a tailored reward function, with the aim of identifying an optimal policy for accurate PR outcome predictions by the RL agent. The sub-questions will further examine the adaptability and performance of the RL agent across various scenarios, including single-stage versus multi-stage reviews, and across two distinct datasets using different RL algorithms.

RQ2: *How do RL approaches compare to baseline and existing models in predicting PR review outcomes?*

This research question aims to validate the effectiveness of our RL approach by comparing its performance against both

2640-7574/24/$31.00 ©2024 IEEE
DOI 10.1109/SANER60148.2024.00057

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:42:59 UTC from IEEE Xplore. Restrictions apply.

a baseline model and predictive models from prior research.

Our work makes the following notable contributions:

- We collect an extensive GitHub dataset comprising of 66,281 PRs and 588,097 review comments, which reflects the dynamics of the PR review process [1].
- We introduce two effective RL-based predictive models for PR outcomes, achieving top G-mean scores of 0.82664 and 0.88372 for different feature sets. These models outperform traditional classifiers in various scenarios, thereby highlighting their robustness. Additionally, we conduct a comprehensive comparative analysis of these RL models against existing predictive approaches, further validating their effectiveness.

The remainder of the paper is organized as follows: Section II reviews relevant literature, Section III provides background on RL and sentiment analysis, and Sections IV details our RL formalizations. Section V presents a comparative analysis, and Section VII concludes with a summary, future work, and threats to validity.

II. Related Work

Given that our research focuses on predicting PR outcomes, and as part of our exploration into different RL configurations for the multi-stage review process, we conduct sentiment analysis to quantify the PR comments. Therefore, we can classify the related work in this study into two groups: *sentiment analysis* in GitHub interactions, and *predictive models* for PR outcomes. On the sentiment analysis front, Guzman et al. [32] used lexical sentiment analysis to explore emotions in commit comments. They found a general trend toward emotional neutrality but observed more negative comments in Java projects and increased negative sentiments on Mondays. Kumar et al. [33] broadened this scope by investigating the impact of sentiment on productivity and code quality across different programming languages, also noting higher negativity on Mondays. Pletea et al. [34] specialized their research on security-related discussions, revealing more negative emotions. Rahman et al. [35] aimed to differentiate between useful and non-useful code review comments, proposing a predictive model that achieved a 66% accuracy rate.

Concerning the second group, predictive models for GitHub PR outcomes, Dey et al. [27] employed a Random Forest model to predict PR acceptance on GitHub. They analyzed 50 PR-related factors from 4,218 popular NPM packages, ultimately narrowing them down to 14 key predictors. With a dataset of 483,988 PRs, their model achieved an AUC-ROC score of 0.95, highlighting its utility for both PR integrators and contributors. In this study, we will refer to this approach as *RForest_PR*.

Jing et al. [28] proposed an XGBoost-based model, CTCPPre, for predicting accepted PRs. Their dataset included 221,096 PRs from 28 GitHub projects and encompassed 17 diverse PR-related factors. The model outperformed existing methods with an AUC-ROC score of 0.76, and an average F1-score of 0.88.

Lastly, Gerede et al. [29] extended the scope to the Gerrit platform[2], exploring various machine learning techniques for predicting code review revisions. Focusing on the Android project, they identified seven key features from 11,633 code reviews. Using multiple algorithms like SVM, AdaBoost, and Random Forest, they found that Random Forest yielded the best performance metrics.

In summary, we observed the following key trends and gaps in the related work. First, existing research on the sentiment analysis of GitHub comments frequently lacked rigorous validation of the sentiment analysis tools used. These tools were not specifically tailored for the technical and informal language commonly found in GitHub comments, such as code snippets and acronyms like "LGTM" (looks good to me). Second, we identified two leading predictive models for PR outcomes— CTCPPre [28] and RForest_PR [27]— that are most closely related to our RL-based approach. While CTCPPre utilizes an XGBoost Classifier without detailing the hyperparameters, RForest_PR employs a fine-tuned Random Forest model, specifying optimal values for parameters like *ntree* and *mtry*. Both of these techniques serve as critical benchmarks in the comparative analysis section of our study. Our study stands at the intersection of these domains, employing RL-based predictive models and integrating sentiment analysis to offer a more comprehensive solution for predicting PR outcomes.

III. Background

A. Reinforcement Learning (RL)

RL focuses on learning optimal decision-making through agent-environment interactions. The agent, as the decision-maker, operates within an environment, taking actions and receiving rewards [36], [37]. The process unfolds in discrete steps, where at each step t:

1) **State and Action.** The agent observes the current state s_t from the state space $\mathcal{S}$, and selects an action a_t from the action space $\mathcal{A}$.
2) **Transition and Reward.** The environment transitions to a new state s_{t+1} and provides a reward r_t based on a function $\mathcal{R}(s_t, a_t)$.
3) **Policy and Optimization.** The agent aims to optimize its policy $\pi(a_t|s_t)$ to maximize the cumulative rewards over time.
4) **Discount Factor (γ) and Episode.** γ balances immediate and future rewards, $\gamma \epsilon (0\ ,\ 1]$. Episodes are sequences of state-action-reward tuples leading to a terminal state.

The state and action spaces can be either discrete or continuous. The agent's goal is to find an optimal policy π that maximizes long-term rewards, where the state transition probabilities $\mathcal{P}(s_{t+1}|s_t, a_t)$ are quantified by the transition function $\mathcal{P}$.

[1] https://zenodo.org/records/10049493

[2] https://www.gerritcodereview.com/

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:42:59 UTC from IEEE Xplore. Restrictions apply.

RL differentiates itself from traditional supervised learning in several key ways. Unlike supervised learning, which relies on predefined input/output pairs to guide the agent, RL agents learn through active exploration of states and actions, receiving feedback via immediate rewards [36]. This enables RL to adapt to complex, sequential decision-making scenarios, where performance evaluation often occurs concurrently with the learning process.

In RL, problems that conform to the Markov property—indicating that state transitions are independent of prior states or agent actions—can be effectively modeled as a Markov Decision Process (MDP). The MDP is mathematically represented by a 5-tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ [37]. MDPs are instrumental in solving RL tasks, especially when the agent must make decisions based on delayed reinforcement. In these scenarios, the rewards for actions may not be immediately apparent, requiring a framework adept at handling long-term consequences and dependencies [36].

Action and Observation Spaces. The action space outlines possible actions an agent can perform, while the observation space specifies what the agent perceives. Not all RL algorithms can handle both discrete and continuous spaces. In the context of our research, we utilize the `StableBaselines3` framework, which building upon OpenAI Baselines, provides a wide range of state-of-the-art RL algorithms, including Advantage Actor-Critic (A2C), Deep Q-Network (DQN), Proximal Policy Optimization (PPO), Deep Deterministic Policy Gradient (DDPG), Soft Actor-Critic (SAC), and Twin Delayed Deep Determinisitc Policy (TD3) [38]. It supports various types of action and observation spaces from OpenAI's `Gym Python` library, such as *Dict, Discrete, Box, MultiDiscrete,* and *MultiBinary* spaces, offering flexibility in algorithm selection based on the problem's specific requirements [39].

B. Sentiment Analysis of PR Comments

In this study, we opted for the Valence Aware Dictionary and Sentiment Reasoner (VADER) [40] for sentiment analysis on GitHub PR comments due to its numerous advantages. VADER [41] uses a universally applicable, human-curated, valence-based sentiment lexicon and combines lexicon and rule-based sentiment analysis approaches. It is adept at capturing linguistic nuances such as negations and emoticons, and operates without the need for training data, significantly reducing pre-processing steps. It provides a nuanced sentiment score, including *positive, negative,* and *neutral* components along with a normalized *compound* score. Its hybrid methodology enables broad applicability across domains, making it well-suited for our diverse set of GitHub in-line code comments. While SentiStrength [42], another lexicon-based tool from existing literature, was considered, it lacked the granularity and extensive operational documentation that VADER offers, making VADER the optimal choice for our sentiment analysis needs in the GitHub PR context.

IV. RL for Predicting PR Outcomes

We approach PR outcome prediction as a decision estimation game, involving an RL agent and the PR environment. While there are multiple ways to formalize PR prediction, we present two distinct methods for modeling the PR review process as an RL problem. The first approach, which we refer to as *BatchRL*, relies on a comprehensive set of features defined based on individual pull requests, and learns to predict review outcomes by analyzing a set of PRs. The second approach, named *ChatRL*, focuses on the interactive discussions occurring within each PR, and learns to predict the review outcomes. It should be noted that *BatchRL* aligns with the established concept of Batch Reinforcement Learning in RL research [43], while *ChatRL* is a novel term introduced in this work to describe our specific methodology. In the following, we provide a detailed formalization of both approaches.

A. BatchRL

In *BatchRL*, the agent, at each time step, observes the current state from a training dataset, train_data$(TD) = \{(x_1, y_1), (x_2, y_2), \ldots, (x_i, y_i)\}$, where x_j is the jth unique PR state and y_j is its expected outcome. In accordance with its policy, the agent selects an action and transitions to the next state, guided by a reward determined by the *merged-or-not* feature, which serves as the ground truth for whether the PR is accepted for merging.

State Space ($\mathcal{S}$): The initial state, denoted as s_1, corresponds to the first sample, x_1, in the training dataset. At each time step t, the state s_t of the environment aligns with the sample x_t. We represent the state space as a high-dimensional feature set that encompasses 67 PR features; the remaining five out of the 72 are project/PR identifiers, which are not included in the state space. Furthermore, the environment is designed to randomize the sample order in each episode, thereby exposing the agent to diverse sequences of PR states and enhancing learning efficiency by preventing the model from overfitting to a specific order.

To handle the high-dimensional state spaces, we leverage the *Gym* library's *Dict* space for multi-feature representation. In this representation, keys correspond to PR features, and their values are specified using the *Box* space, which covers all possible feature ranges.

Action Space ($\mathcal{A}$): We used a *Discrete* action space with two actions: $\{0, 1\}$ corresponding to PR rejection and acceptance, respectively. The binary action space aligns with the dichotomous nature of the *merged-or-not* feature in the dataset.

Episode: We defined an episode as tuples (s, a, r), where s is the state, a is the action taken, and r is the immediate reward. An episode concludes either when all samples in the training set are exhausted or upon incorrect prediction of a PR rejection, which is the minority outcome in our dataset; this is done deliberately due to the imbalanced dataset with PR rejection as the minority class action.

Reward Function $\mathcal{R}(s_t, a_t)$: We initially used a simple reward function, offering a reward of +1 for *correct* predictions and -1 for *incorrect* ones. However, this led to a biased model

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:42:59 UTC from IEEE Xplore. Restrictions apply.

due to the imbalanced dataset, necessitating a revision. Thus, we revised the function to use a cost-sensitive tuning approach, incorporating the Imbalance Ratio (IR) — the ratio of minority to majority class instances — to penalize incorrect predictions of the minority class more heavily. The final rewards are set to $+1$ or -1 for PR rejection and $+k$ or $-k$ for PR acceptance, where $k \in [0, 1)$ is the trade-off parameter. Multiple k values are explored during training to find the most effective trade-off.

B. ChatRL

This approach integrates raw comments from developers and reviewers to provide the RL agent with nuanced insights into sentiments and opinions, enhancing prediction accuracy. In contrast to our previous formalization, which relies on a broad array of PR features, this method underscores the importance of PR discussions in decision-making. It introduces notable variations in both the observation space and episode configurations, thereby providing a unique angle for predicting PR outcomes.

In this RL formalization, each episode corresponds to the review process of a single PR, where the RL agent sequentially encounters in-line comments. The sequence of comments, dictated by timestamps, forms the review stages for the PR. The agent makes decisions to either *accept* or *reject* the PR at each review stage based on the current comment. The environment then offers immediate, neutral rewards and advances the agent to the next state, representing the subsequent comment in the review sequence. This continues until the end of the episode, resulting in a final reward based on the accuracy of the agent's overall decision. The reward function uses the *merged-or-not* feature to assess the correctness of the agent's actions.

In summary, the formalized training dataset is represented as train_data$(TD) = \{(x_1, y_1), (x_2, y_2), \ldots, (x_i, y_i)\}$, where each x_j signifies the jth PR in the dataset, and each y_j indicates the expected outcome of the respective PR review. Each x_j can be described as a sequence of PR review stages, $x_j = \{c_1, c_2, \ldots, c_n\}$, where c_n denotes the review stage for the nth inline comment in the jth PR. This notation encapsulates the essence of our RL formalization for the PR comments dataset, which we name PRChat dataset, aiming for an efficient and insightful decision-making process.

State Space $\mathcal{S}$: The definition of the state space is grounded in the PR comment features extracted from the training dataset. The initial state, denoted as s_1, is determined by the features of the first PR, x_1, encapsulating its various review stages $\{c_1, c_2, \ldots, c_n\}$. To efficiently represent this diverse set of PR comment features, we employ the *Dict* type from the `Gym` library, which effectively covers the entire range of PR comment features.

Action Space $\mathcal{A}$: The binary outcome of a pull request review is straightforward, representing either acceptance (coded as 1) or rejection (coded as 0). This binary structure is implemented using the *Discrete* type from the `Gym` library, which allows for the clear distinction between these two possible review outcomes.

Reward Function $\mathcal{R}(s_t, a_t)$. Zero reward is assigned for each step t; non-zero rewards are given only at the end of the episode based on the final prediction. The reward function is cost-sensitive, with $k = 2 * IR$ serving as the trade-off parameter.

V. Evaluation

This section describes the empirical evaluation of our approach, evaluation metrics, baselines, datasets, research questions, experiments, and results.

A. Evaluation Metrics

To assess the performance of baseline models and RL agents, we employ a range of metrics suited for imbalanced datasets, as shown in Table I: Accuracy (ACC), F1-score, Geometric Mean (G-mean), and AUC-ROC. While accuracy is a commonly used measure, it may not adequately capture performance nuances in imbalanced datasets [44], [45], [46]. Hence, we augment it with F1-score, G-mean, and AUC-ROC, which collectively offer a more granular evaluation and a comprehensive understanding of our predictive model's performance in the context of imbalanced GitHub PR datasets [47]. The F1-Score, G-Mean Score, and AUC-ROC are metrics that range from 0 to 1, where higher values generally indicate better classification performance.

B. Baselines

To evaluate our RL-based predictive model, we compare it against established methods. Naive Bayes is selected as a simple baseline. Meanwhile, Random Forest and XGBoost serve as more advanced benchmarks, being the underlying implementations in the PR outcome prediction techniques RForest_PR and CTCPPre, respectively, as identified in the Related Work (Section II). These algorithms are adjusted for class imbalance using either Random Under-Sampling (RUS), Random Over-Sampling (ROS), or by employing the dataset in its original, unaltered form [44], [48].

Naive Bayes [49], [50]. Serves as a simple baseline, using conditional independence and Bayes' theorem for prediction.

Random Forest [51], [52], [50]. An ensemble of decision trees, known for robustness and scalability but considered a black-box model.

XGBoost [53]. An advanced ensemble method with iterative error correction, optimized for speed and scalability.

Sampling Methods. To address class imbalance, we use RUS and ROS techniques, targeting an adjusted IR of 0.5. The justification for this target is that any improvement in IR beyond 0.5 leads to drastic changes in the datasets, such as a large drop in total datapoints when using RUS and a high number of repetitions when using ROS. Specifically, the IR is approximately 0.12 in the comprehensive PR features dataset, which we refer to as PRBatch, and approximately 0.26 in the second PR comments dataset, named PRChat. RUS balances classes by removing instances from the majority class, while ROS balances classes by duplicating instances from the minority class.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:42:59 UTC from IEEE Xplore. Restrictions apply.

Metric	Description	Metric	Description
Accuracy	Defined as $ACC = \frac{TP+TN}{TP+TN+FP+FN}$	G-mean	Measures the balance in predicting both the PR acceptance and rejection classes and is defined as, $G\text{-mean} = \sqrt{\frac{TP}{TP+FN} \times \frac{TN}{TN+FP}}$
F1-score	Balances precision and recall, crucial for imbalanced datasets. It is defined as, $F1 = \frac{2 \times \frac{TP}{TP+FP} \times \frac{TP}{TP+FN}}{\frac{TP}{TP+FP} + \frac{TP}{TP+FN}}$	AUC-ROC	Quantifies the model's ability to discriminate between positive and negative samples across varying thresholds.
Confusion Matrix	Breaks down model predictions into True Positives (TP), False Negatives (FN), False Positives (FP), and True Negatives (TN), serving as the foundation for other metrics. In our models, a True Positive (TP) occurs when a PR review is correctly predicted as accepted (class-1), and a True Negative (TN) when correctly predicted as rejected (class-0). A False Positive (FP) arises if a PR review is wrongly predicted as accepted, while a False Negative (FN) occurs when it is incorrectly predicted as rejected.		

TABLE I: Comprehensive List of Evaluation Metrics and Descriptions

C. Datasets

Our study uses two datasets *PRBatch* and *PRChat* for training of agents with formalization BatchRL (Section IV-A) and ChatRL (Section IV-B) respectively.

1) PRBatch Dataset: PRBatch uses an extensive dataset prepared by Zhang et al. (2021) [3]. This dataset was selected for its diversity in project activity, language, and size, offering a more generalizable and holistic view of PR dynamics across various software development scenarios. In contrast to several existing studies that rely on the limited dataset by Gousios et al. (2014) [1], which we observed frequently in our literature review, Zhang et al.'s dataset is substantially richer, containing 12 times more projects and 10 times more PRs. The dataset is further enhanced by addressing missing values and introducing new features like *same_user*, indicating whether the contributor and integrator are the same individual, making it particularly suitable for our research.

We performed necessary pre-processing steps mainly handling missing values by replacing negative and missing values with *Not a Number (NaN)* and omitting features with over 30% missing values. We chose a 30% threshold to balance data integrity with maximizing the number of retained PRs after feature omission.

In terms of feature engineering, redundant features were removed and related features like *files-added* and *files-deleted* were consolidated into *files-changed*. Rather than narrowing down key variables, our study aims to showcase the adaptability of RL algorithms in handling extensive feature sets; therefore, we retain a large number of features in the dataset. Correct data types were set for each feature, and categorical values in the *language* feature were label-encoded. IR was calculated for the target feature *merged-or-not*, guiding the model design. Initial correlation analysis using the Phi Coefficient and Point-biserial Correlation Coefficient [54] revealed minimal interdependencies among features, therefore eliminating the need for a comprehensive correlation analysis. The selection of these specific coefficients was informed by established methods commonly used in the literature on similar topics, ensuring methodological rigor [3], [8]. Finally, the dataset was randomly split into 80% training and 20% testing sets, retaining 1,045,883 and 260,195 PRs, respectively.

2) PRChat Dataset: In addition to PRBatch, we also leveraged the PRChat dataset, focusing on in-line code comments within GitHub PRs for predicting outcomes. Initially, we intended to directly download comments for the PRs in the PRBatch dataset using the GitHub REST API. However, due to incomplete responses and the extensive data cleaning required for ensuring completeness and accuracy, we opted to start with an existing work. We sourced the raw comments from the work by Sinha et al. (2021) [55], who used GHArchive [56] to collect a vast set of 54,021,838 PR review comments from January 2015 to December 2020. These comments were specifically tailored to include in-line code comments, facilitated through the *PR Review Comment Event* in the GitHub API. To enhance data quality, we filtered the dataset to include only comments in English, as the sentiment analysis tool VADER does not inherently provide its own translation. We further augmented the dataset with additional features like *owner-name*, *repo-name*, *pull-no*, and *merged-or-not* using the GitHub REST APIs. These identifier features are crucial for leveraging GitHub APIs to collect more features for our study.

We then augmented the dataset with features extracted from raw PR comments as well as sentiment scores. Since ChatRL relies solely on comments for predicting PR outcomes, we employed sentiment analysis to quantify the raw comments. Through careful feature extraction and sentiment analysis, we captured the full context and nuances within each comment. These enhancements significantly contribute to the dataset's overall depth and comprehensiveness, making it well-suited for sophisticated PR outcome prediction.

For the feature extraction process, we aimed to capture various dimensions of the PR comments that could be indicative of their nature and ultimately the PR's outcome. We utilized `Regular Expression (regex)` to identify code suggestions within the comments, resulting in a binary feature named *has-code-element* that indicates the presence or absence of code elements. We counted valid English words using the `PyEnchant Python` package, excluding punctuation and special characters, and named this feature *word-count*. This feature provides insights into the verbosity

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:42:59 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Training the RL-based PR Outcome Prediction Model

Input : Training dataset $TD = \{(pr_1, o_1), (pr_2, o_2), ..., (pr_i, o_i)\}$
/* PR state-outcome pairs, outcome is the expected action set from the *merged-or-not* PR feature */
Input : No. of PR samples in training data = i
Input : total_timesteps $T = 1,400,000$
Output: Trained RL-based PR prediction model

Initialize the RL environment
  Set the action space $A = \{0, 1\}$ // Actions: 0 (reject), 1 (accept)
Set the observation space $\mathcal{O}$ with the selected PR features as described in TD
  Set $step_t = 1$ // Number of timesteps
Set $E = 0$ // Number of Episodes
while $step_t < T$ **do**
  Call reset function to reset the environment for a new episode
    Set $t = 1$ // Step within the episode
  Set initial state $s_1 = pr_1$
  repeat
    Select action a_t using RL policy π: $a_t = \pi(s_t)$;
    Perform action a_t in the environment to get the next state s_{t+1} and the immediate reward r_t;
    Update the RL model using the observed reward r_t: UpdateModel(s_t, a_t, r_t, s_{t+1});
    Set $s_t = s_{t+1}$;
    Increment t: $t = t + 1$;
    Increment $step_t$: $step_t = step_t + 1$;
  until $t > i$ *or* $r_t == -1$;
  Increment E: $E = E + 1$;
end while
Return the trained RL-based PR prediction model

of each comment, which could correlate with the detail and thoroughness of the review. Additionally, we calculated the proportion of stop words, which are common words deemed insignificant for natural language data processing, to total words in each comment using the `nltk` library, and named this feature *stopw-ratio*.

For sentiment analysis, we employed VADER to add individual emotion scores (*'pos-vr', 'neg-vr', 'neu-vr'*) and a *compound* score to the dataset. A temporary categorical *emotion* feature, based on *compound* score thresholds, was introduced for the manual validation of VADER's performance. We conducted a manual review of a balanced subset of 200 comments, comparing them with VADER's classifications, and found that despite VADER being primarily trained on social media data and potentially less adept at handling technical jargon in GitHub comments, it achieved a G-mean score of 0.7542, and an accuracy of 74%. To address its limitations, we incorporated the previously discussed *has_code_element* feature. Supported by studies in Section II that successfully used similar tools on GitHub comments, led us to focus on the granular positive, negative, and neutral scores for model training, excluding the *emotion* feature whose sole purpose was manual validation. This methodology justified our use of VADER for sentiment analysis in this context.

The final PRChat dataset comprised of 588,097 comments related to 66,281 different PRs and incorporated 15 distinct PR characteristics. We split the data into training and testing sets (80/20) for model evaluation. Preliminary correlation analysis did not reveal strong associations between the *merged-or-not* feature and other features, suggesting that each feature contributes independently to PR outcome prediction.

D. RL Algorithm Selection

Based on the compatibility assessment concerning the action and observation space in `StableBaselines3`, where the choice of algorithms depends on the action space configuration, the A2C, DQN, and PPO algorithms were shortlisted for their suitability with our defined state and action spaces, crucial for effectively handling the task of PR outcome prediction.

E. Research Questions

RQ1: How feasible and effective are RL approaches in predicting outcomes of PR reviews?

This research question explores the practicality and performance of the two proposed RL formalization approaches, treating them as separate sub-questions detailed below.

1) RQ1.1: Can a BatchRL agent accurately predict PR review outcome?: In this section, we examine the effectiveness of BatchRL agent (Section IV-A) in predicting PR review outcomes. We leverage the PRBatch dataset (Section V-C1), which is a large-scale, comprehensive dataset of PR features.

Training of RL Agents. The training process of BatchRL agents follows the steps outlined in Algorithm 1. At the beginning of each episode, the reset function is called, which involves shuffling the PR samples and initializing the agent's starting state. Throughout the training, the agent continually engages with the environment, refining its policy based on immediate rewards and state transitions. Episodes come to a close under two conditions: either after reviewing all the PR samples or upon making an incorrect prediction leading to a PR rejection.

We employ the `StableBaselines3` library's default parameters to train our BatchRL agents using the DQN, A2C, and PPO algorithms. The models are trained through the `learn()` method, utilizing a total of 1,400,000 timesteps, which approximates the total number of PRs in the dataset, ensuring comprehensive training. Evaluation is carried out using the `predict()` method with the deterministic attribute set to `False`, ensuring no policy updates during testing.

Model Architectures. The neural network architectures differ slightly among the DQN, PPO, and A2C algorithms. For DQN, we employ a two-layer neural network with 64 units in each layer, activated by the `ReLU` function. This design helps capture complex relationships within the 67 input features from the dataset. In contrast, both PPO and A2C utilize a two-layer neural network but opt for the `Tanh` activation function. These layers, also containing 64 units each, are followed by a Softmax layer to handle discrete predictions for the PR outcomes. These architectural choices are tailored

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:42:59 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Performance of A2C and PPO Algorithms for PR Outcome Prediction with Different Values of Trade-Off Factor (k)

A2C					PPO				
	F-1 Score		G-mean	AUC-ROC		F-1 Score		G-mean	AUC-ROC
	Class-0	Class-1				Class-0	Class-1		
k = 0.05*IR	0.20	0.00	0.00360	0.50000	k = 0.5*IR	0.20	0.13	0.25860	0.51943
k = 0.5*IR	0.23	0.91	0.44632	0.56552	k = 1*IR	0.22	0.48	0.51107	0.56594
k = 1*IR	0.00	0.94	0.00596	0.50001	k = 3*IR	0.24	0.90	0.47725	0.57636

- 80/20 split: train.shape (1045883, 72), i.e., 1045883 samples with 72 features, and test.shape (260195, 72)
- IR = 0.12 - total_timesteps (T) = 1,400,000 - Class-0: PR Rejection and Class-1: PR Acceptance

TABLE III: Performance of DQN Algorithm for PR Outcome Prediction with Different Values of Trade-Off Factor (k)

total_timesteps (T) = 1,400,000	F-1 Score		G-mean	AUC-ROC	Overall Accuracy (%)
	Class-0	Class-1			
k = 2*IR	0.35	0.81	0.72359	0.72398	71
k = 3*IR	0.35	0.80	0.71878	0.71980	69
k = 4*IR	0.29	0.87	0.58809	0.62242	78

- 80/20 split: train.shape (1045883, 72) and test.shape (260195, 72) - IR = 0.12

to the unique requirements of each algorithm and are aimed at optimizing performance on our PR outcome prediction task.

Handling Class Imbalance and Parameter Tuning. To address the imbalanced nature of the dataset, we implement a cost-sensitive reward function. The sensitivity of the model towards minority classes is fine-tuned by varying the values of k based on the dataset's IR. The initial range for the trade-off parameter k is set as $\{0.05 * IR, 0.5 * IR, 1 * IR, 2 * IR, 3 * IR, 4 * IR, 5 * IR\}$. For the initial training of each RL algorithm, we start with a subset of k values, specifically $k = \{0.1 * IR, 0.5 * IR, 1 * IR, 5 * IR\}$. Based on the performance of these initial runs, the set of k values is then updated for each algorithm for further analysis and validation.

To achieve optimal performance, we employ a two-step iterative refinement of the model. Initially, we focus on determining the optimal values for the trade-off parameter, guided by preliminary evaluation results. Subsequently, we adjust the *total_timesteps* , with its optimal value discussed in the results section. This iterative process ensures that both the trade-off parameter and *total_timesteps* are finely tuned for best performance.

RQ1.1 Results. Initial tests with varying k values revealed that A2C and PPO algorithms exhibited performance biases towards either the majority or minority class, depending on the trade-off factor. Due to consistent underperformance across all k values, we discontinued using A2C for further analysis. Preliminary tests for PPO also showed subpar G-mean scores, indicating challenges in class balance. In contrast, DQN demonstrated superior and consistent performance, particularly for $k = 2 * IR$, making it the preferred choice for further investigation. The top three performance evaluations for each algorithm are presented in Table II for A2C and PPO, and in Table III for DQN.

In our pursuit to fine-tune the DQN model for optimal PR outcome prediction, we also conducted an in-depth analysis to identify the best *total_timesteps* value under different data splits, while holding the trade-off factor constant at $k = 2 * IR$. In the 80/20 data split, our analysis covered a range of *total_timesteps* from 1,400,000 up to 3,500,000. The optimal value was 2,300,000, which outperformed other configurations in terms of G-mean, AUC-ROC, and F-1 score. For the 50/50 data split, we started at 700,000 *total_timesteps* and incremented by 100,000, up to 2,000,000. The best-performing configuration was at 1,900,000 timesteps. Lastly, in a 20/80 split, we found 1,400,000 to be the optimal *total_timesteps*, starting from 300,000 and evaluating up to 1,500,000. These results underscore both the DQN algorithm's adaptability and the pivotal role of the *total_timesteps* parameter, affirming its impact on model performance and its sensitivity to the dataset's size and distribution. The implications of our RQ1.1 results challenge the need for exhaustive hyperparameter tuning. We observed that simply optimizing *total_timesteps* and using a problem-specific reward function yielded strong performance. This could streamline the RL problem-solving process, emphasizing the role of problem definition and specialized reward functions. The findings also highlight untapped potential in further problem-specific optimizations, despite not yet delving into neural network tuning.

RQ1.1 Using BatchRL approach, DQN achieved the best G-mean score of 0.82664 with an 80/20 split and outperformed A2C and PPO at $k = 2 * IR$. Optimal total_timesteps(T) for DQN were $T = 2,300,000$ for an 80/20 split, $T = 1,900,000$ for a 50/50 split, and $T = 1,400,000$ for a 20/80 split.

2) RQ1.2: Can a ChatRL agent accurately predict PR review outcome?: In this section, we explore our second RL formalization, ChatRL (outlined in Section IV-B), which predicts PR outcomes by exclusively focusing on GitHub PR discussions. Unlike the BatchRL formalization that utilizes a wide array of PR features, ChatRL emphasizes the critical role of PR discussions in decision-making.

Training of RL Agents. In RQ1.2, we primarily focus on DQN due to its exceptional performance in RQ1.1; however, we also investigate the A2C and PPO algorithms using the `StableBaselines3` framework. To recall: (i) the state and action spaces of ChatRL are of *Dict* and *Discrete* types,

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:42:59 UTC from IEEE Xplore. Restrictions apply.

TABLE IV: Performance of A2C and PPO Algorithms for Outcome Prediction with PR Comments Dataset

A2C			PPO		
total_timesteps (T)	**G-mean**	**AUC-ROC**	**total_timesteps (T)**	**G-mean**	**AUC-ROC**
T = 700,000	0.01903	0.50	T = 1,200,000	0.52474	0.55530
T = 800,000	0.0	0.50	T = 1,400,000	0.43600	0.51476
T = 1,000,000	0.01901	0.50	T = 1,500,000	0.48931	0.51580

- 80/20 split: train.shape (471831, 15), and test.shape (116266, 15) - IR = 0.26, k = 1*IR

TABLE V: Performance of DQN Algorithm for Outcome Prediction with PR Comments Dataset

total_timesteps (T)	**Accuracy (%)**			**F-1 Score**		**G-mean**	**AUC-ROC**
	Class-0	**Class-1**	**Overall**	**Class-0**	**Class-1**		
T = 800,000	83	94	92	0.81	0.95	0.88372	0.88565
T = 1,000,000	34	80	75	0.32	0.85	0.49459	0.57812
T = 1,500,000	52	45	49	0.29	0.60	0.49336	0.49338

- 80/20 split: train.shape (471831, 15) and test.shape (116266, 15) - IR = 0.26, k = 2*IR

respectively, and (ii) the PRChat dataset (Section V-C2) serves for training and includes 471,831 in-line comments from 53,000 unique PRs, with an IR of 0.26.

In the training procedure, the initialization step involves shuffling the PRs at the start of each episode. The initial state is then set by taking the first PR and arranging its associated review stages and in-line comments in chronological order, with the initial state corresponding to the earliest timestamped review stage within that reordered PR. Following this, the agent enters an action-reward loop where it predicts PR outcomes and receives zero immediate reward; this process continues until all PR review stages are covered, concluding the episode. At the end of each episode, the agent's final prediction earns a non-zero reward based on its accuracy. Throughout training, the RL model aims to maximize aggregate rewards to refine its prediction policy.

Model Architecture. The neural network architecture for DQN employs the `MultiInputPolicy` class and features an input layer with eight dimensions corresponding to the PR comment features. It includes two fully connected hidden layers, each with 64 output features, and utilizes the `ReLU` activation function. The output layer generates a 2-element vector for predicting PR outcomes. The architecture is optimized using the `Adam` optimizer.

Handling Class Imbalance and Parameter Tuning. Given the IR of 0.26, the training process addresses class imbalance using the trade-off factor k. We also vary the *total_timesteps* in experiments to determine the optimal value for this parameter. The training employs default parameters and the *total_timesteps* count is initially set at 600,000, corresponding to the total number of PR comments in the dataset.

RQ1.2 Results. The top three performances for each algorithm are summarized in Tables IV and V. The dataset was split 80/20 for training and testing. Based on prior findings in Section V-E1, we set the trade-off factor in the cost-sensitive reward function as $k = 2 * IR$ for DQN and $k = 1 * IR$ for A2C and PPO. For $k = 2 * IR$, A2C and PPO yielded a G-mean score of 0.0, failing to predict the minority class.

A2C and PPO showed a strong bias towards the majority class, yielding poor G-mean scores even after adjusting the trade-off factor. In contrast, DQN outperformed both. Our experiments initially set the *total_timesteps* parameter at 600,000. We then incremented the *total_timesteps* by 100,000, up to 1,500,000, for further analysis. The optimal value for *total_timesteps* with DQN was found to be 800,000, achieving a top G-mean score of 0.88372. Therefore, we proceed with DQN for further analysis, fixing *total_timesteps* at 800,000 and $k = 2 * IR$.

RQ1.2 findings highlight DQN's adaptability in multi-stage review processes. Notably, we found that incorporating individual sentiment scores for positive, negative, and neutral comments instead of a single sentiment score mitigated some limitations and biases inherent in sentiment analysis tools like VADER. While this emphasizes RL's potential for nuanced modeling in multi-stage reviews, the study is exploratory and focuses on demonstrating RL's adaptability in predicting PR outcomes at each stage.

RQ1.2 Using ChatRL formalization, the optimum value of the total_timesteps (T) parameter for DQN with $k = 2 *$ IR for an 80/20 data split is $T = 800,000$ yielding a G-mean Score of 0.88372.

F. RQ2: How do RL approaches compare to baseline and existing models in predicting PR review outcomes?

In Section V-B, we designate Naive Bayes as the baseline model and use Random Forest and XGBoost as benchmarks for comparison. These models were selected due to their role as the underlying techniques for existing PR outcome predictive models, namely CTCPPre [28] and RForest_PR [27], which we introduced for comparative analysis in Section II. Hereafter, *Batch-DQN* refers to the RL formalization following the BatchRL approach using DQN algorithm and utilizing the PRBatch dataset, as detailed in Section V-E1. Similarly, *Chat-DQN* refers to the RL formalization following the ChatRL approach using DQN algorithm and utilizing the PRChat dataset, as detailed in Section V-E2.

Comparative Analysis: Batch-DQN. Using an 80/20 data split for the training and testing datasets, we employed ROS and RUS to adjust the class imbalance and compared the performance of Random Forest and XGBoost classifiers with

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:42:59 UTC from IEEE Xplore. Restrictions apply.

TABLE VI: Comparative Performance of Predictive Models with 80/20 Data Split and Resampling Techniques

		Batch-DQN[a]	Baseline (Naive Bayes)	RForest_PR [27]	CTCPPre [28]	Random Forest		XGBoost	
						ROS*	RUS**	ROS*	RUS**
Accuracy (%)		84	75	91	92	92	49	89	49
F-1 Score	Class-0	0.53	0.25	0.43	0.48	0.48	0.26	0.56	0.25
	Class-1	0.91	0.85	0.95	0.95	0.95	0.61	0.94	0.61
G-mean		0.82664	0.54526	0.54270	0.59715	0.59769	0.60409	0.76631	0.59908
AUC-ROC		0.82691	0.58600	0.64355	0.67271	0.67314	0.63214	0.7795	0.62335

80/20 split: train.shape (1045883, 72) and test.shape (260195, 72) Class-0: PR Rejection and Class-1: PR Acceptance
[a] For Batch-DQN, IR = 0.12, k = 2*IR, total_timesteps = 2,300,000
* For ROS, IR = 0.5 and train.shape: (1398807, 72)
** For RUS, IR = 0.5 and train.shape: (340035, 72)

TABLE VII: Comparison of Impact of Training Sample Size on Predictive Model Performance

Data Split			Batch-DQN*	Baseline (Naive Bayes)	Random Forest	XBGoost
80/20[a]	Accuracy (%)		84	75	91	92
	F-1 Score	Class-0	0.53	0.25	0.43	0.48
		Class-1	0.91	0.85	0.95	0.95
	G-mean		0.82664	0.54526	0.54270	0.59715
	AUC-ROC		0.82691	0.58600	0.64355	0.67271
50/50[b]	Accuracy (%)		74	76	91	92
	F-1 Score	Class-0	0.42	0.25	0.40	0.48
		Class-1	0.84	0.86	0.95	0.95
	G-mean		0.79132	0.55100	0.51848	0.59459
	AUC-ROC		0.79394	0.59149	0.63056	0.67112
20/80[c]	Accuracy (%)		63	76	91	91
	F-1 Score	Class-0	0.34	0.26	0.37	0.47
		Class-1	0.74	0.86	0.95	0.95
	G-mean		0.72284	0.55119	0.49010	0.58921
	AUC-ROC		0.73582	0.59254	0.61651	0.66756

* For Batch-DQN, IR = 0.12 and k = 2*IR, total_timesteps = T
[a] train.shape (1045883, 72) and test.shape (260195, 72) while IR = 0.12, T = 2,300,000
[b] train.shape (656007, 72) and test.shape (650071, 72) while IR = 0.12, T = 1,900,000
[c] train.shape (260195, 72) and test.shape (1045883, 72) while IR = 0.12, T = 1,400,000

our RL-based Batch-DQN model. ROS inflated the minority class (PR rejection) from 113,345 to 466,269 instances, while RUS reduced the majority class (PR acceptance) from 932,538 to 226,690. Given the drastic impact on dataset size, we limited the adjusted IR to 0.5; Batch-DQN still outperformed other methods (Table VI), achieving a G-mean score of 0.82664.

Impacts of Training Sample Size. We also examine the impact of training sample size on predictive model performance using various training and testing splits (80/20, 50/50, and 20/80). In all scenarios, our Batch-DQN model consistently outperformed other methods. With an 80/20 split, the Batch-DQN model achieved G-mean and AUC-ROC scores nearing 0.82664 (Table VII). Similarly, for a 50/50 split, the scores were around 0.79132, and for a 20/80 split, they were approximately 0.72284, all with an IR of 0.12.

To address the occasionally higher F1-scores from Random Forest and XGBoost in different data splits—despite their lower G-mean and AUC-ROC scores compared to our RL-based model—we conducted further analysis. Our RL model shows a more balanced performance in handling both PR acceptance and rejection actions, validated by higher G-mean scores in Table VI and Table VII. Specifically, we examined the 50/50 and 20/80 training/testing splits with testing datasets sized (650071, 72) and (1045883, 72) respectively. After segmenting these based on the *merged-or-not* feature, Batch-DQN achieved an 86% accuracy in predicting PR rejections specifically. This significantly outperformed Random Forest and XGBoost, which managed accuracies of 27% and 3% respectively, emphasizing the value of using the G-mean score for model evaluation.

Comparative Analysis: Chat-DQN. We faced unique challenges in developing our second RL-based Chat-DQN model for predicting PR outcomes based on individual review stages, using only in-line code comments. This approach is pioneering, with limited existing research for reference. While some studies employed sentiment analysis on GitHub comments, they did not focus on PR outcome prediction. Our model fills this gap, offering a novel tool for project management and code review optimization. This scarcity of relevant work led us to retain Naive Bayes as our baseline and Random Forest and XGBoost as benchmarks, all implemented in Python. To address dataset imbalance, we applied RUS and ROS, normalizing the IR to 0.5 for improved performance.

We split the dataset 80/20 for training and testing, implemented three prediction techniques, and summarized the results in Table VIII. As shown, Chat-DQN comfortably

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:42:59 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII: Comprehensive Comparative Performance Analysis of All Predictive Models

		Chat-DQN[a]	Baseline (Naive Bayes)	Random Forest	XGBoost	Random Forest		XGBoost	
						ROS*	RUS**	ROS*	RUS**
Accuracy (%)		92	77	78	79	76	74	79	79
F-1 Score	Class-0	0.81	0.06	0.11	0.05	0.15	0.18	0.07	0.07
	Class-1	0.95	0.87	0.87	0.88	0.86	0.84	0.88	0.88
G-mean		0.88454	0.17558	0.24756	0.16540	0.31320	0.34863	0.19289	0.19916
AUC-ROC		0.88642	0.50217	0.51629	0.51281	0.51831	0.51737	0.51437	0.51404

80/20 split: train.shape (471831, 15) and test.shape (116266, 15) while IR = 0.26
[a] For Chat-DQN, IR = 0.26 and trade-off factor (k) = 2*IR, total_timesteps (T) = 800,000
* For ROS, IR = 0.5 and train.shape: (557065, 15)
** For RUS, IR = 0.5 and train.shape: (301361, 15)

outperformed other approaches, achieving a G-mean score of 0.88454.

Implications and Insights. Our RQ2 findings affirm the RL models, Batch-DQN and Chat-DQN, as superior to existing benchmarks, particularly in predicting PR rejections. Batch-DQN can be implemented in existing workflows to flag PRs with higher chances of rejection. Interestingly, Chat-DQN, designed for multi-stage review and relying only on PR discussions, outperformed Batch-DQN in G-mean scores. This highlights RL's adaptability to complex review settings and stresses the G-mean score's importance as a robust evaluation metric over mere accuracy rates.

VI. Threats to Validity

Our work, though grounded in extensive prior research on pull-based development, inherits certain limitations, which are outlined below:

Internal Threats. Choices made during *Data Preprocessing* could impact the representativeness of our dataset, thereby affecting the generalizability of our predictive models. Despite utilizing extensive datasets and diverse test cases, the possibility of *Data Bias* remains; intrinsic biases may still exist within the data, potentially limiting the generalizability of our models. *Hyperparameter Tuning* is another concern: the study lacks exhaustive hyperparameter optimization, which might constrain the optimal performance of our models.

External Threats. *Platform-Specificity* arises as our data is sourced exclusively from GitHub, potentially limiting the external validity of our findings when applied to other coding platforms like Gerrit and GitLab. *Sentiment Analysis* also presents a concern, as the limitations of the VADER framework used could impact the accuracy of our second RL formalization.

In summary, these limitations are integral to our study and guide future research efforts. While rigorous design minimizes their impact, acknowledging these constraints provides a pathway for refining methodologies in subsequent RL-based PR prediction studies.

VII. Conclusion

This study pioneers the use of RL for predicting PR outcomes through two distinct formalizations. The first, employing a dataset of over 1.3 million PRs with 72 features, showcased DQN's adaptability and accuracy with G-mean scores ranging from 0.72284 to 0.82664 across different data splits and set new benchmarks in PR rejection predictions with accuracies up to 87%. The second formalization focused on PR discussions using a dataset of about 0.6 million comments from 66,281 PRs, achieved a G-mean of 0.88454, and significantly outperformed traditional classifiers like Random Forest and XGBoost, with the closest competitor achieving a G-mean of 0.34863. Comparative analysis confirmed DQN's robustness, particularly in handling data imbalances through optimized trade-off factors and cost-sensitive reward functions. These two RL formalizations offer contrasting insights: the comprehensive model provides a broad lens, while the discussions-based approach focuses on the nuances of developer interactions, setting a new performance benchmark. The study not only advances predictive modeling and collaboration in software development but also provides a rich groundwork for future research by publishing a curated PR Comments Dataset and making all model source code available[3].

While this study marks an advancement in using RL for PR outcome predictions, several avenues for future exploration remain open. Beyond the cost-sensitive reward function employed in this study, the integration of advanced resampling techniques like SMOTE [44], [46] or ADASYN [57] as well as the exploration of RL as an alternative to these techniques for generating synthetic data could be investigated for more effective management of imbalanced datasets. Another dimension for exploration is the expansion of the current action space, which is limited to PR acceptance and rejection. Future studies could incorporate more nuanced actions, such as marking for changes or engaging in discussions, guided by the insights from this work. Modifying the existing RL formalization could also yield an agent capable of making targeted suggestions to reviewers, thereby improving the review process. Lastly, considering more complex actions necessitates the use of specialized datasets, possibly focusing on specific, active GitHub repositories to better emulate human review processes.

Acknowledgement. This work has been supported by the Natural Sciences and Engineering Research Council of Canada (NSERC).

[3]https://github.com/kernish/RL-PullRequest-Predictions

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:42:59 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] G. Gousios, M. Pinzger, and A. v. Deursen, "An exploratory study of the pull-based software development model," in *Proceedings of the 36th International Conference on Software Engineering*, ser. ICSE 2014. New York, NY, USA: Association for Computing Machinery, 2014, p. 345–355. [Online]. Available: https://doi.org/10.1145/2568225.2568260

[2] O. Kononenko, T. Rose, O. Baysal, M. Godfrey, D. Theisen, and B. de Water, "Studying pull request merges: A case study of shopify's active merchant," in *2018 IEEE/ACM 40th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*, 2018, pp. 124–133.

[3] X. Zhang, Y. Yu, G. Georgios, and A. Rastogi, "Pull request decisions explained: An empirical overview," *IEEE Transactions on Software Engineering*, pp. 1–1, 2022.

[4] "GitHub," https://octoverse.github.com/, [Accessed 15-01-2024].

[5] G. Kudrjavets, A. Kumar, N. Nagappan, and A. Rastogi, "Mining code review data to understand waiting times between acceptance and merging: An empirical analysis," in *Proceedings of the 19th International Conference on Mining Software Repositories*, ser. MSR '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 579–590. [Online]. Available: https://doi.org/10.1145/3524842.3528432

[6] O. Kononenko, O. Baysal, and M. W. Godfrey, "Code review quality: How developers see it," in *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*, 2016, pp. 1028–1038.

[7] C. Maddila, C. Bansal, and N. Nagappan, "Predicting pull request completion time: A case study on large scale cloud services," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2019. New York, NY, USA: Association for Computing Machinery, 2019, p. 874–882. [Online]. Available: https://doi.org/10.1145/3338906.3340457

[8] O. Baysal, O. Kononenko, R. Holmes, and M. W. Godfrey, "Investigating technical and non-technical factors influencing modern code review," *Empirical Software Engineering*, vol. 21, no. 3, pp. 932–959, Jun 2016. [Online]. Available: https://doi.org/10.1007/s10664-015-9366-8

[9] D. M. Soares, M. L. de Lima Júnior, L. Murta, and A. Plastino, "Acceptance factors of pull requests in open-source projects," in *Proceedings of the 30th Annual ACM Symposium on Applied Computing*, ser. SAC '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 1541–1546. [Online]. Available: https://doi.org/10.1145/2695664.2695856

[10] P. Pooput and P. Muenchaisri, "Finding impact factors for rejection of pull requests on GitHub," in *Proceedings of the 2018 VII International Conference on Network, Communication and Computing*, ser. ICNCC 2018. New York, NY, USA: Association for Computing Machinery, 2018, p. 70–76. [Online]. Available: https://doi.org/10.1145/3301326.3301380

[11] V. Lenarduzzi, V. Nikkola, N. Saarimäki, and D. Taibi, "Does code quality affect pull request acceptance? an empirical study," *Journal of Systems and Software*, vol. 171, p. 110806, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121220302090

[12] M. C. Silva, M. T. Valente, and R. Terra, "Does technical debt lead to the rejection of pull requests?" in *Proceedings of the XII Brazilian Symposium on Information Systems on Brazilian Symposium on Information Systems: Information Systems in the Cloud Computing Era - Volume 1*, ser. SBSI 2016. Porto Alegre, BRA: Brazilian Computer Society, 2016, p. 248–254.

[13] J. Tsay, L. Dabbish, and J. Herbsleb, "Influence of social and technical factors for evaluating contribution in GitHub," in *Proceedings of the 36th International Conference on Software Engineering*, ser. ICSE 2014. New York, NY, USA: Association for Computing Machinery, 2014, p. 356–366. [Online]. Available: https://doi.org/10.1145/2568225.2568315

[14] R. N. Iyer, S. A. Yun, M. Nagappan, and J. Hoey, "Effects of personality traits on pull request acceptance," *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2632–2643, 2021.

[15] G. Gousios, M.-A. Storey, and A. Bacchelli, "Work practices and challenges in pull-based development: The contributor's perspective," in *Proceedings of the 38th International Conference on Software Engineering*, ser. ICSE '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 285–296. [Online]. Available: https://doi.org/10.1145/2884781.2884826

[16] G. Gousios, A. Zaidman, M.-A. Storey, and A. v. Deursen, "Work practices and challenges in pull-based development: The integrator's perspective," in *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 1, 2015, pp. 358–368.

[17] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, "Modern code review: A case study at google," in *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*, ser. ICSE-SEIP '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 181–190. [Online]. Available: https://doi.org/10.1145/3183519.3183525

[18] E. van der Veen, G. Gousios, and A. Zaidman, "Automatically prioritizing pull requests," in *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*, 2015, pp. 357–361.

[19] M. B. Zanjani, H. Kagdi, and C. Bird, "Automatically recommending peer reviewers in modern code review," *IEEE Transactions on Software Engineering*, vol. 42, no. 6, pp. 530–543, 2016.

[20] G. Zhao, D. A. da Costa, and Y. Zou, "Improving the pull requests review process using learning-to-rank algorithms," *Empirical Software Engineering*, vol. 24, no. 4, pp. 2140–2170, Aug 2019. [Online]. Available: https://doi.org/10.1007/s10664-019-09696-8

[21] C. Maddila, S. S. Upadrasta, C. Bansal, N. Nagappan, G. Gousios, and A. van Deursen, "Nudge: Accelerating overdue pull requests toward completion," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 2, mar 2023. [Online]. Available: https://doi.org/10.1145/3544791

[22] J. Tsay, L. Dabbish, and J. Herbsleb, "Let's talk about it: Evaluating contributions through discussion in GitHub," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. FSE 2014. New York, NY, USA: Association for Computing Machinery, 2014, p. 144–154. [Online]. Available: https://doi.org/10.1145/2635868.2635882

[23] J. Zhu, M. Zhou, and A. Mockus, "Effectiveness of code contribution: From patch-based to pull-request-based tools," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. FSE 2016. New York, NY, USA: Association for Computing Machinery, 2016, p. 871–882. [Online]. Available: https://doi.org/10.1145/2950290.2950364

[24] O. Kononenko, O. Baysal, L. Guerrouj, Y. Cao, and M. W. Godfrey, "Investigating code review quality: Do people and participation matter?" in *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2015, pp. 111–120.

[25] Z.-X. Li, Y. Yu, G. Yin, T. Wang, and H.-M. Wang, "What are they talking about? analyzing code reviews in pull-based development model," *Journal of Computer Science and Technology*, vol. 32, no. 6, pp. 1060–1075, Nov 2017. [Online]. Available: https://doi.org/10.1007/s11390-017-1783-2

[26] Y. Yu, G. Yin, T. Wang, C. Yang, and H. Wang, "Determinants of pull-based development in the context of continuous integration," *Science China Information Sciences*, vol. 59, no. 8, p. 080104, Jul 2016. [Online]. Available: https://doi.org/10.1007/s11432-016-5595-8

[27] T. Dey and A. Mockus, "Which pull requests get accepted and why? A study of popular NPM packages," *CoRR*, vol. abs/2003.01153, 2020. [Online]. Available: https://arxiv.org/abs/2003.01153

[28] J. Jiang, J.-t. Zheng, Y. Yang, and L. Zhang, "Ctcppre: A prediction method for accepted pull requests in GitHub," *Journal of Central South University*, vol. 27, no. 2, pp. 449–468, Feb 2020. [Online]. Available: https://doi.org/10.1007/s11771-020-4308-z

[29] Ç. E. Gerede and Z. Mazan, "Will it pass? predicting the outcome of a source code review," *Turkish Journal of Electrical Engineering and Computer Sciences*, vol. 26, no. 3, pp. 1343–1353, 2018.

[30] M. Bagherzadeh, N. Kahani, and L. Briand, "Reinforcement learning for test case prioritization," *IEEE Transactions on Software Engineering*, vol. 48, no. 8, pp. 2836–2856, 2022.

[31] Y. Wan, Z. Zhao, M. Yang, G. Xu, H. Ying, J. Wu, and P. S. Yu, "Improving automatic source code summarization via deep reinforcement learning," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, ser. ASE '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 397–407. [Online]. Available: https://doi.org/10.1145/3238147.3238206

[32] E. Guzman, D. Azócar, and Y. Li, "Sentiment analysis of commit comments in GitHub: An empirical study," in *Proceedings of the 11th Working Conference on Mining Software Repositories*, ser. MSR 2014. New York, NY, USA: Association for Computing Machinery, 2014, p. 352–355. [Online]. Available: https://doi.org/10.1145/2597073.2597118

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:42:59 UTC from IEEE Xplore. Restrictions apply.

[33] A. Kumar, M. Khare, and S. Tiwari, "Sentiment analysis of developers' comments on GitHub repository: A study," in *2022 14th International Conference on Advanced Computational Intelligence (ICACI)*, 2022, pp. 91–98.
[34] D. Pletea, B. Vasilescu, and A. Serebrenik, "Security and emotion: Sentiment analysis of security discussions on GitHub," in *Proceedings of the 11th Working Conference on Mining Software Repositories*, ser. MSR 2014. New York, NY, USA: Association for Computing Machinery, 2014, p. 348–351. [Online]. Available: https://doi.org/10.1145/2597073.2597117
[35] M. M. Rahman, C. K. Roy, and R. G. Kula, "Predicting usefulness of code review comments using textual features and developer experience," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*, 2017, pp. 215–226.
[36] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *Journal of artificial intelligence research*, vol. 4, pp. 237–285, 1996.
[37] Y. Li, "Deep reinforcement learning," 2018.
[38] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: http://jmlr.org/papers/v22/20-1364.html
[39] "Spaces -Gym Documentation- gymlibrary.dev," https://www.gymlibrary.dev/api/spaces/, [Accessed 15-08-2023].
[40] C. Hutto and E. Gilbert, "Vader: A parsimonious rule-based model for sentiment analysis of social media text," *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 8, no. 1, pp. 216–225, May 2014. [Online]. Available: https://ojs.aaai.org/index.php/ICWSM/article/view/14550
[41] "VADER," https://github.com/cjhutto/vaderSentiment, [Accessed 15-01-2024].
[42] "SentiStrength," http://sentistrength.wlv.ac.uk/, [Accessed 15-01-2024].
[43] S. Lange, T. Gabel, and M. Riedmiller, *Batch Reinforcement Learning*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 45–73. [Online]. Available: https://doi.org/10.1007/978-3-642-27645-3_2
[44] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: synthetic minority over-sampling technique," *Journal of artificial intelligence research*, vol. 16, pp. 321–357, 2002.
[45] F. Zampetti, G. Bavota, G. Canfora, and M. D. Penta, "A study on the interplay between pull request review and continuous integration builds," in *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2019, pp. 38–48.
[46] J. Mendoza, J. Mycroft, L. Milbury, N. Kahani, and J. Jaskolka, "On the effectiveness of data balancing techniques in the context of ml-based test case prioritization," in *Proceedings of the 18th International Conference on Predictive Models and Data Analytics in Software Engineering*, 2022, pp. 72–81.
[47] V. López, A. Fernández, S. García, V. Palade, and F. Herrera, "An insight into classification with imbalanced data: Empirical results and current trends on using data intrinsic characteristics," *Information Sciences*, vol. 250, pp. 113–141, 2013. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0020025513005124
[48] V. Kumar, G. S. Lalotra, P. Sasikala, D. S. Rajput, R. Kaluri, K. Lakshmanna, M. Shorfuzzaman, A. Alsufyani, and M. Uddin, "Addressing binary classification over class imbalanced clinical datasets using computationally intelligent techniques," *Healthcare*, vol. 10, no. 7, 2022. [Online]. Available: https://www.mdpi.com/2227-9032/10/7/1293
[49] H. Zhang, "The optimality of naive bayes," *Aa*, vol. 1, no. 2, p. 3, 2004.
[50] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
[51] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct 2001. [Online]. Available: https://doi.org/10.1023/A:1010933404324
[52] R. CouronnÃ©, P. Probst, and A.-L. Boulesteix, "Random forest versus logistic regression: a large-scale benchmark experiment," *BMC Bioinformatics*, vol. 19, no. 1, p. 270, Jul 2018. [Online]. Available: https://doi.org/10.1186/s12859-018-2264-5
[53] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 785–794. [Online]. Available: https://doi.org/10.1145/2939672.2939785
[54] D. Kornbrot, "Point biserial correlation," Wiley StatsRef: Statistics Reference, 2014.
[55] A. Sinha, "Pull request review comments dataset," May 2021.
[56] "GHArchive," https://www.gharchive.org/.
[57] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in *2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*, 2008, pp. 1322–1328.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:42:59 UTC from IEEE Xplore. Restrictions apply.